CONFIDENTIAL · SECURITY AUDIT REPORT

Security Audit Tech for Palestine Website

techforpalestine.org · Cloudflare Pages & Workers

AUDIT DATE	SCOPE	STACK
2026-03-31	API Routes · Middleware · CF Worker · Config	Astro v5 · React 19 · Cloudflare Pages · Notion API

Executive Summary

Senior security review · All files in `src/pages/api/`, `src/middleware/`, `cloudflare-worker/`, `src/store/`, `src/utils/`, and configuration files.

3	5	6	3
CRITICAL	HIGH	MEDIUM	LOW

The most critical issues require **immediate action before the next deployment**. A live API key is committed to version control and must be considered fully compromised. Two API endpoints allow unauthenticated, rate-unlimited attackers to send fraudulent membership invitations and trigger privileged backend actions without any payment or verification.

The remaining findings — while serious — can be remediated in a planned sprint. Several good practices are already in place and are documented at the end of this report.

OWASP TOP 10 MAPPING

Finding	Severity	OWASP Category
C-1 · API Key in Version Control	CRITICAL	A02 · Cryptographic Failures
C-2 · Unauthenticated Invite Endpoint	CRITICAL	A01 · Broken Access Control
C-3 · Unverified Webhook Triggers Invites	CRITICAL	A01 · Broken Access Control
H-1 · Secret Key Logged to Browser Console	HIGH	A09 · Security Logging Failures
H-2 · PII Echoed Back in Response Body	HIGH	A02 · Cryptographic Failures
H-3 · Middleware Caches API POST Responses	HIGH	A05 · Security Misconfiguration
H-4 · No Rate Limiting on Any Endpoint	HIGH	A04 · Insecure Design

Finding	Severity	OWASP Category
H-5 · Wildcard CORS on All Endpoints	HIGH	A05 · Security Misconfiguration
M-1 · No CSRF Protection	MEDIUM	A01 · Broken Access Control
M-2 · No Input Length / Format Validation	MEDIUM	A03 · Injection
M-3 · getEnv Falls Back to globalThis	MEDIUM	A03 · Injection
M-4 · Verbose Logging of S3 Signed URLs	MEDIUM	A09 · Security Logging Failures
M-5 · Error Details Leaked in Responses	MEDIUM	A05 · Security Misconfiguration
M-6 · Missing Security Headers (CSP etc.)	MEDIUM	A05 · Security Misconfiguration
L-1 · CF Account ID in Version Control	LOW	A05 · Security Misconfiguration
L-2 · External Script Without SRI	LOW	A08 · Software & Data Integrity
L-3 · Image Proxy Has No Auth	LOW	A01 · Broken Access Control

CRITICAL FINDINGS

CRITICAL C-1

API Key and Secrets Hardcoded in Version-Controlled File

File: wrangler.toml:17 · cloudflare-worker/wrangler.toml:4

OWASP: A02:2021 – Cryptographic Failures · A05:2021 – Security Misconfiguration

The `PROJECTHUB_API_KEY` (a 64-character bearer token) and the Cloudflare `account_id` are committed in plaintext to tracked files that are visible in git history, diffs, and any repository clone.

```
# wrangler.toml:17 — LIVE IN GIT HISTORY
PROJECTHUB_API_KEY="823522de634eb8852314bf236326611c9a5a72b158d3e6325b4bc5cfa
a5e949a"

# cloudflare-worker/wrangler.toml:4
account_id = "cf1bd4ae16c96d5e9bf794817c06f3cc"
```

IMPACT

Any person with read access to the repository possesses a valid credential to the ProjectHub API. The account ID aids in targeted social engineering against Cloudflare support.

ATTACK SCENARIO

Attacker clones the repo, extracts the key, calls `projecthub.techforpalestine.org` with full API-level privileges (read member data, mutate projects), or sells the credential.

SECURE FIX

1. Rotate the key immediately — the credential in git history is permanently compromised.

2. Use Cloudflare Wrangler secret management:

```
wrangler secret put PROJECTHUB_API_KEY
wrangler secret put HUB_API_KEY
```

3. Remove the `[vars]` block from `wrangler.toml`. Add a `wrangler.toml.example` with placeholder values, and add both files to `.gitignore`.

4. Scrub the history: `git filter-repo --path wrangler.toml --invert-paths`

CRITICAL C-2

Unauthenticated Membership Invitation Endpoint

File: src/pages/api/membership-invite.ts **OWASP:** A01:2021 – Broken Access Control

The `/api/membership-invite` endpoint is a completely open proxy to the Hub API's invitation system. It requires no authentication, no CSRF token, no rate limiting, and no request validation. It blindly forwards any JSON body to the privileged Hub `/api/auth/invite` endpoint using the server-held bearer token.

```
// membership-invite.ts:25-32 — anyone on the internet can call this
const upstream = await fetch(`${hubApiUrl}/api/auth/invite`, {
  method: "POST",
  headers: {
    Authorization: `Bearer ${hubApiKey}`,  // your secret, their request
  },
  body: JSON.stringify(body),  // entirely attacker-controlled
});
```

ATTACK SCENARIO

```
curl -X POST https://techforpalestine.org/api/membership-invite \
  -H "Content-Type: application/json" \
  -d '{"email": "victim@example.com", "type": "paid"}'
```

SECURE FIX

Eliminate this public endpoint entirely. The Hub API should only be called from authenticated server-side logic (e.g., the qgiv webhook after payment verification). If it must exist, require a shared secret header validated server-side and add strict IP allowlisting or per-IP rate limiting.

CRITICAL C-3

Unverified Webhook Triggers Privileged Membership Invitation

File: src/pages/api/qgiv-webhook.ts:10–56

OWASP: A01:2021 – Broken Access Control · A07:2021 – Identification & Authentication Failures

The QGiv webhook endpoint accepts any POST request. If the payload contains `formId=1116610` and `isRecurring=y`, it immediately calls the Hub API to issue a paid membership invitation — with no signature verification, no IP allowlisting, and no authentication. QGiv supports HMAC-SHA256 webhook signing; it is not used.

```
// qgiv-webhook.ts:17-22 — all fields are attacker-controlled
const formId = payload["formId"] ?? payload["Form Id"] ?? null;
const isMembershipForm = String(formId) === MEMBERSHIP_FORM_ID;
const isRecurring = payload["isRecurring"] === "y";

if (isMembershipForm && isRecurring) {
  // invite email from payload["contactEmail"] — zero verification
```

ATTACK SCENARIO

```
curl -X POST https://techforpalestine.org/api/qgiv-webhook \
  -H "Content-Type: application/json" \
  -d '{"formId":"1116610","isRecurring":"y","contactEmail":"attacker@evil.co
m"}'
```

SECURE FIX

Implement QGiv webhook signature verification. QGiv sends an HMAC-SHA256 signature in a request header:

```
const signature = request.headers.get("X-QGiv-Signature");
const body = await request.text();
const expected = hmac("sha256", QGIV_WEBHOOK_SECRET, body, "hex");

// Use timing-safe comparison to prevent timing attacks
if (!timingSafeEqual(Buffer.from(signature), Buffer.from(expected))) {
  return new Response("Unauthorized", { status: 401 });
}
```

Alternatively, allowlist QGiv's published IP ranges at the Cloudflare WAF level.

HIGH FINDINGS

HIGH H-1 Secret Key Logged to Client-Side Browser Console

File: src/store/api.ts:7–8

OWASP: A09:2021 – Security Logging Failures · A02:2021 – Cryptographic Failures

`api.ts` is imported by client-side React form components. At module load time it logs `PUBLIC_SECRET_KEY` to the console. This key is also embedded in client-side Axios headers on every form submission, making it readable to any user via browser DevTools.

```
// api.ts:7-8 — executes in the browser bundle
console.log("PUBLIC_API_URL:", API_URL);
console.log("PUBLIC_SECRET_KEY:", SECRET_KEY);  // visible in DevTools
```

SECURE FIX

Remove all `console.log` calls from production paths. Proxy all authenticated requests through a server-side Astro API route that holds the credential exclusively server-side. Rename `PUBLIC_SECRET_KEY` — the `PUBLIC_` prefix in Astro signals intentional client-side exposure and is semantically wrong for a credential.

HIGH H-2

Full Webhook Payload (Including PII) Echoed in HTTP Response

File: src/pages/api/qgiv-webhook.ts:153–164 **OWASP:** A02:2021 – Cryptographic Failures

When donation type cannot be determined, the raw webhook payload — containing donor name, email, address, and transaction ID — is returned verbatim in the HTTP response body, and separately logged in full on line 14.

```
// qgiv-webhook.ts:153-164
return new Response(JSON.stringify({
  success: true,
  message: "Webhook received but donation type unclear",
  payload,  // ← full unredacted QGiv payload with PII
}), ...);
```

SECURE FIX

Never echo back raw external payloads. Return only `{"success": true}`. Replace the full payload `console.log` with structured logging that redacts email, name, and payment fields.

HIGH H-3 Global Middleware Caches POST API Responses as "public"

File: src/middleware/index.ts:6 **OWASP:** A05:2021 – Security Misconfiguration

The global middleware unconditionally applies `Cache-Control: public, max-age=600` to every response. POST endpoints like `/api/endorsement-request` and `/api/e4p-pledge-sign` do not override this. A CDN or intermediate proxy could cache a 201 response and silently discard future submissions.

```
// middleware/index.ts:6 — no path or method filtering
response.headers.set("Cache-Control", "public, max-age=600");
```

SECURE FIX

```
export const onRequest = defineMiddleware(async (context, next) => {
  const response = await next();
  const isApi = context.url.pathname.startsWith("/api/");
  const isGet = context.request.method === "GET";

  response.headers.set(
    "Cache-Control",
    !isGet || isApi ? "no-store" : "public, max-age=600"
  );
  return response;
});
```

HIGH H-4 No Rate Limiting on Any Endpoint

File: All files in src/pages/api/ **OWASP:** A04:2021 – Insecure Design

Every API endpoint — including those that write to Notion databases, send emails, and trigger Hub invitations — has zero rate limiting. An attacker can flood write endpoints to fill Notion with spam entries or generate mass invitation emails at no cost.

SECURE FIX

Implement Cloudflare WAF rate limiting rules (free tier) scoped to POST requests on `/api/*`. Typical threshold: 10 requests / 60 seconds per IP for write endpoints. Additionally, add honeypot fields to all forms and reject submissions where the honeypot is filled.

HIGH H-5

Wildcard CORS on All API Endpoints

File: Every file in src/pages/api/ **OWASP:** A05:2021 – Security Misconfiguration

Every API endpoint returns `Access-Control-Allow-Origin: *`. Combined with missing CSRF protection (M-1), this allows any malicious website to make cross-origin AJAX requests to write endpoints on behalf of a visiting user.

SECURE FIX

Restrict CORS on write endpoints to the known origin:

```
"Access-Control-Allow-Origin": "https://techforpalestine.org"
```

Wildcard CORS is acceptable for read-only public data endpoints (GET `/api/events`, etc.) but should be an explicit, documented decision, not a default applied everywhere.

MEDIUM FINDINGS

MEDIUM M-1 No CSRF Protection on State-Mutating Endpoints

File: src/pages/api/endorsement-request.ts · src/pages/api/e4p-pledge-sign.ts

OWASP: A01:2021 – Broken Access Control

Forms submit JSON to POST endpoints without CSRF tokens. Any third-party page can use `fetch()` to submit these forms on behalf of a victim's browser. The wildcard CORS policy (H-5) amplifies the risk by removing the same-origin barrier.

> **SECURE FIX**
>
> Verify the `Origin` header on all POST endpoints matches `https://techforpalestine.org`. For cookie-based sessions, use `SameSite=Strict`. For stateless JSON APIs, the `Origin` check is the most practical mitigation.

MEDIUM M-2

No Input Length or Format Validation on Form Endpoints

File: src/pages/api/endorsement-request.ts · src/pages/api/e4p-pledge-sign.ts

OWASP: A03:2021 – Injection

Presence checks exist, but there is no validation of email format, URL format (website, campaign link, LinkedIn accept arbitrary strings), or string length limits. An attacker can submit a 10 MB `campaignPurpose` field or a `javascript:` URI as a "URL".

SECURE FIX

```
// Add server-side validation to endorsement-request.ts
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(endorsementData.contactEmail)) { /* 400 */ }

try { new URL(endorsementData.organizationWebsite); }
catch { /* return 400 — invalid URL */ }

if (endorsementData.campaignPurpose.length > 2000) { /* 400 */ }
```

MEDIUM M-3

getEnv Falls Back to Unsafe globalThis Property Lookup

File: src/utils/getEnv.ts:24–26 **OWASP:** A03:2021 – Injection

The final fallback in `getEnv` reads arbitrary properties off `globalThis` by variable name. If any third-party code or prototype pollution attack sets `globalThis.NOTION_SECRET`, it will be used as the Notion API credential.

```
// getEnv.ts:24-26 — prototype pollution risk
if (typeof globalThis !== "undefined" && (globalThis as any)[name]) {
  return (globalThis as any)[name];
}
```

SECURE FIX

Remove this fallback entirely. Cloudflare runtime `locals.runtime.env` and `import.meta.env` are the only two legitimate sources in this stack.

MEDIUM M-4

Verbose Logging of AWS S3 Signed URLs and Internal Objects

File: src/store/notionClient.ts:45–68 · src/utils/imageProxy.ts:54

OWASP: A09:2021 – Security Logging Failures

Multiple `console.log` calls emit full Notion file objects, AWS S3 signed URLs (which contain embedded AWS credentials in query parameters: `X-Amz-Credential`, `X-Amz-Signature`), and proxy URL mappings. In any centralised log aggregation system, these logs constitute a data leakage risk.

SECURE FIX

Remove all debug `console.log` calls from production paths. Keep only `console.error` for genuine runtime exceptions, and ensure error objects are sanitised before logging (strip URLs with credentials).

MEDIUM M-5

Internal Error Details Returned in HTTP Response Bodies

File: src/pages/api/qgiv-webhook.ts:138 · :172–173 **OWASP:** A05:2021 – Security Misconfiguration

Internal error messages (including exception `.message` strings from fetch failures) are returned in HTTP response bodies. These may include internal hostnames, library versions, or structural information useful to an attacker during reconnaissance.

```
// qgiv-webhook.ts:172
details: error instanceof Error ? error.message : "Unknown error",
```

SECURE FIX

Log full errors server-side with a correlation ID. Return only a generic message to the client: `{"error": "An internal error occurred", "ref": "<correlationId>"}`

MEDIUM M-6

Missing Security Headers (CSP, X-Frame-Options, etc.)

File: astro.config.mjs · src/middleware/index.ts **OWASP:** A05:2021 – Security Misconfiguration

No security headers are set anywhere in the application. The absence of a Content Security Policy is the most impactful gap — it leaves the site open to XSS escalation and inline script injection with no browser-level mitigation.

SECURE FIX — ADD TO MIDDLEWARE OR PUBLIC/_HEADERS

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
Content-Security-Policy: default-src 'self';
  script-src 'self' https://secure.qgiv.com;
  img-src 'self' data: https:;
  connect-src 'self' https://plausible.io;
```

LOW FINDINGS

LOW L-1 Cloudflare Account ID Committed to Version Control

File: cloudflare-worker/wrangler.toml:4 **OWASP:** A05:2021 – Security Misconfiguration

The Cloudflare `account_id` is not a secret per Cloudflare's own model, but its presence in a public repository confirms infrastructure relationships and aids targeted social engineering. Move to an environment variable and add the file to `.gitignore`.

LOW L-2 Externally-Loaded Payment Script Without Subresource Integrity

File: src/components/MembershipPage.tsx **OWASP:** A08:2021 – Software & Data Integrity Failures

The QGiv embed script is dynamically injected from `https://secure.qgiv.com/resources/core/js/embed.js` without an `integrity` attribute. A compromised QGiv CDN could serve malicious JavaScript in the payment context.

SECURE FIX

```
<script
  src="https://secure.qgiv.com/resources/core/js/embed.js"
  integrity="sha384-<compute-from-known-good-version>"
  crossorigin="anonymous"></script>
```

LOW L-3 Notion Image Proxy Has No Authentication or Access Control

File: cloudflare-worker/worker.js:41–62 **OWASP:** A01:2021 – Broken Access Control

The worker validates that decoded URLs point to Notion S3 domains (preventing open-relay abuse), but any actor knowing the worker URL can use it to fetch and cache any valid Notion S3 URL they possess, potentially bypassing Notion's own expiry mechanisms for content they should no longer have access to. Low severity given the images are already public via Notion's CDN.

GOOD PRACTICES OBSERVED

- **Approval gating on signatories:** `e4p-signatories.ts` filters on `Approved: true` — new pledge submissions are not immediately public.
- **Server-side credential handling:** Notion secrets are read from `locals.runtime.env` (Cloudflare bindings) in production API routes — not from client-side code (the `api.ts` file is the exception).
- **S3 URL expiry detection:** The image proxy worker correctly detects and handles expired S3 presigned URLs rather than caching broken image responses.
- **HTTP method restriction:** The Cloudflare Worker returns HTTP 405 for non-GET/HEAD/OPTIONS methods.
- **Generic error messages:** Most API routes return generic failure messages without internal exception details (except the qgiv webhook as noted in M-5).
- **Notion write validation:** Required-field presence checks exist on all form submission endpoints before writing to Notion.

PRIORITISED REMEDIATION PLAN

Priority	Action	Finding	Effort
Immediate	Rotate `PROJECTHUB_API_KEY` and `HUB_API_KEY` — both are permanently compromised	C-1	Minutes
Immediate	Move all secrets from `wrangler.toml` to Cloudflare secret bindings; add file to `.gitignore`	C-1	1 hour

Priority	Action	Finding	Effort
Day 1	Remove or gate the `/api/membership-invite` endpoint — it is an unauthenticated invite proxy	C-2	1 hour
Day 1	Add QGiv webhook HMAC signature verification before processing any webhook payload	C-3	2 hours
Day 1	Remove all `console.log` calls from `api.ts`, `notionClient.ts`, and `imageProxy.ts`	H-1, M-4	30 min
Week 1	Fix global middleware to not set `public` cache on POST or API responses	H-3	30 min
Week 1	Implement Cloudflare WAF rate limiting on all POST endpoints (10 req / 60 s per IP)	H-4	2 hours
Week 1	Restrict CORS to `techforpalestine.org` on all write endpoints; retain wildcard only for public read APIs	H-5	1 hour
Week 1	Add security headers (CSP, X-Frame-Options, X-Content-Type-Options) via middleware or `_headers`	M-6	1 hour
Week 1	Stop returning full payload / internal error messages in HTTP responses	H-2, M-5	30 min
Week 2	Add email format, URL format, and length validation to all form endpoints	M-2	3 hours
Week 2	Add `Origin` header check to all POST endpoints for CSRF mitigation	M-1	1 hour
Week 2	Remove `globalThis` property-name fallback from `getEnv`	M-3	15 min
Week 2	Add SRI integrity hash to QGiv embed script; scrub CF account ID from version control	L-1, L-2	30 min